PX4 Python SITL: Dynamics, Vehicle Models, Sensors, Forces, and External Interfaces

PX4 Python SITL

July 8, 2026

Contents

1 Scope

This document describes the simulation kernel and interfaces implemented in the repository:

- 6DOF rigid-body kernel and integration flow,
- vehicle model parameterizations (`x8`, `iris`, `ts06`),
- force and moment models per vehicle,
- sensor models (IMU, magnetometer, barometer, GPS),
- communication with PX4 over MAVLink and with external programs over UDP/WebSocket.

Frames used are NED for navigation and FRD for body axes.

2 Variable Inventory

Table **??** lists symbols used in equations, including units and dimensions.

Table 1: Variables, default values, units, and dimensions

Symbol	Meaning	Default	Unit	Dimension
y	full simulation state vector	set by initial condition and integration	mixed	mixed
$\dot{y}$	state derivative vector	computed each step	mixed	mixed
$p_{ned} = [p_n, p_e, p_d]^T$	position in local North-East-Down frame	initial $[0, 0, -3]$ in main loop	m	L
$v_b = [u, v, w]^T$	body-frame linear velocity (FRD)	initial $[0, 0, 0]$	m/s	LT^{-1}
v_{ned}	NED velocity	derived from $R_{bw} v_b$	m/s	LT^{-1}
$\omega_b = [p, q, r]^T$	body-frame angular velocity	initial $[0, 0, 0]$	rad/s	T^{-1}
$q = [q_w, q_x, q_y, q_z]^T$	attitude quaternion used in kernel	default identity $[1, 0, 0, 0]$	1	1
$R_{wb}(q)$	world-to-body rotation matrix	derived from q	1	1
R_{bw}	body-to-world rotation matrix (R_{wb}^T)	derived from q	1	1
$f_b = [F_x, F_y, F_z]^T$	net body force	sum of active and passive models	N	MLT^{-2}
$m_b = [M_x, M_y, M_z]^T$	net body moment	sum of active and passive models	N m	ML^2T^{-2}
I	inertia matrix at center of gravity	vehicle specific (Iris: diag(5*e*-3, 5*e*-3, 9*e*-3))	kg m^2	ML^2
m	vehicle mass	vehicle specific (Iris: 1.5)	kg	M
g	gravity magnitude	9.81	m/s^2	LT^{-2}
g_w	gravity vector in world frame for IMU model	$[0, 0, -9.8068]$	m/s^2	LT^{-2}
a_b	body translational acceleration	computed each step	m/s^2	LT^{-2}
$a_{imu,in}$	acceleration fed to IMU model	$\dot{v}_b + \omega_b \times v_b$	m/s^2	LT^{-2}
f_{spec}	specific force before IMU filtering/noise	computed each step	m/s^2	LT^{-2}
τ_a, τ_g	IMU filter time constants	from $f_c = 120$ Hz ($\tau \approx 1.326$ ms)	s	T
$f_{c,acc}, f_{c,gyro}$	IMU LPF cutoff frequencies	$120, 120$	Hz	T^{-1}
$f_{imu}, f_{mag}, f_{baro}, f_{gps}$	sensor update rates (IMU/Mag/Baro/GPS)	$250, 100, 20, 5$	Hz	T^{-1}

Symbol	Meaning	Default	Unit	Dimension
α_a, α_g	LPF update coefficients	computed from Δt and τ	1	1
b_a, b_g, b_m	accel/gyro/mag bias states	initialized from sensor bias settings	sensor unit	sensor dimension
n_a, n_g, n_m, n_P, n_q	additive white-noise terms	zero mean (noise enabled)	sensor unit	sensor dimension
$\sigma_{acc}, \sigma_{gyro}$	IMU continuous noise densities	from IMU defaults	$\mathrm{m/s^2}/\sqrt{\mathrm{Hz}}$, $\mathrm{rad/s}/\sqrt{\mathrm{Hz}}$	sensor dimension $T^{-1/2}$
$\sigma_{a,d}, \sigma_{\omega,d}$	IMU discrete noise std. dev.	$\sigma/\sqrt{\Delta t}$	$\mathrm{m/s^2}$, rad/s	sensor dimension
$\sigma_{ba,d}, \sigma_{bg,d}, \sigma_{bm,d}$	discrete bias-innovation std. dev. (acc/gyro/mag)	computed from rw and τ_c	sensor unit	sensor dimension
$rw_{acc}, rw_{gyro}, rw_{mag}$	bias random-walk amplitudes	defaults in IMU/Mag classes	sensor unit$\sqrt{\mathrm{Hz}}$	sensor dimension $T^{-1/2}$
σ_{mag}, σ_P	magnetometer/barometer noise level	defaults in sensor classes	gauss/$\sqrt{\mathrm{Hz}}$, Pa	sensor dimension
$\sigma_{xy}, \sigma_z, \sigma_{vxy}, \sigma_{vz}$	GPS pos/vel noise densities	defaults in GPS class	m/$\sqrt{\mathrm{Hz}}$, m/s/$\sqrt{\mathrm{Hz}}$	sensor dimension $T^{-1/2}$
ϕ_a, ϕ_g, ϕ_m	Gauss-Markov persistence terms	$\exp(-\Delta t/\tau_c)$	1	1
$\eta_{ba}, \eta_{bg}, \eta_{bm}$	Gauss-Markov innovation terms	zero mean Gaussian draws	sensor unit	sensor dimension
w_i	GPS bias-driving random-walk term ($i \in \{N, E, D\}$)	sampled Gaussian process	m/s	LT^{-1}
$m_{ned} = [m_N, m_E, m_D]^T$	Earth magnetic field vector in NED	model default $[0.215, 0.010, 0.430]$	gauss	magnetic flux density
H, X, Y, Z	intermediate magnetic components used to form m_{ned}	computed from δ, ι, B_0	gauss	magnetic flux density
δ, ι, B_0	magnetic declination, inclination, field strength	$0.0391, 1.1345, 0.475$	rad,rad,gauss	1,1,magnetic flux density
S_{si}	soft-iron calibration matrix	identity I_3	1	1
b_{hi}	hard-iron offset vector	from model mag bias (default zero)	gauss	magnetic flux density
z_{local}	barometer altitude input	$-p_d$	m	L
h_0	home altitude (AMSL)	GPS model set-home altitude (default 488, often overridden by model origin)	m	L
h_{amsl}	current altitude above mean sea level	$h_0 + z_{local}$	m	L
T_0, T	ISA reference/current temperature	$T_0 = 288.15$, T computed	K	Θ
P_0, P_{ideal}, P_{meas}	pressure terms in barometer model	$P_0 = 101325$, others computed	Pa	$ML^{-1}T^{-2}$
ρ_{air}, ρ_{baro}	air density terms	$\rho_{air} = 1.225$, ρ_{baro} computed	$\mathrm{kg/m^3}$	ML^{-3}
L	ISA temperature lapse rate	0.0065	K/m	ΘL^{-1}
d_P	accumulated barometer pressure drift	initialized 0	Pa	$ML^{-1}T^{-2}$
r_{drift}	barometer pressure drift rate	SensorSuite default 0.05	Pa/s	$ML^{-1}T^{-3}$
R_E	Earth radius in GPS reprojection	6,371,000	m	L
φ, λ	latitude and longitude	computed by reprojection	rad	1
φ_0, λ_0	GPS origin latitude/longitude	47.397742°, 8.545594°	rad	1
$\beta_N, \beta_E, \beta_D$	GPS position random-walk states	initialized 0	m	L

Symbol	Meaning	Default	Unit	Dimension
τ_{gps}	GPS bias/random-walk correlation time	60	s	T
$\nu_\cdot, \xi_\cdot$	GPS position and velocity noise terms	zero mean, configured by noise densities	m or m/s	L or LT^{-1}

3 Simulation Kernel

3.1 Short Introduction

The rigid-body kernel in `src/dynamics/dynamics.py` evolves the 13-state vector under the summed body-frame force and moment input and gravity. Figure **??** introduces the coordinate conventions used by the equations: world frame NED (North-East-Down) and body frame FRD (Forward-Right-Down).

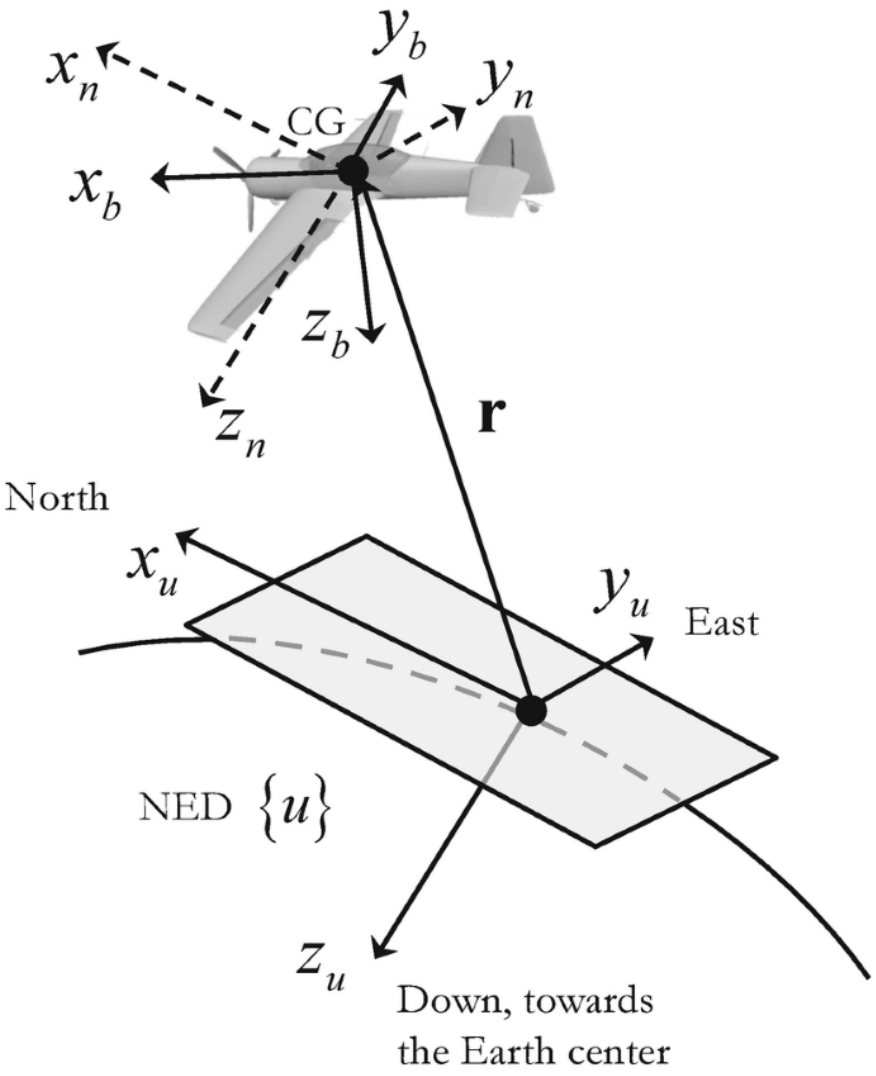


Figure 1: Coordinate systems used in the simulator: local NED world frame and FRD body frame.

3.2 6DOF Model

3.2.1 Assumptions

- Constant mass and inertia over one simulation step.
- Forces and moments are fully summed in body frame before the dynamics call.
- Ground-contact clamp at $z_{ned} = 0$: if $p_d \geq 0$, then $p_d \leftarrow 0$, and downward components are clamped by $v_{ned,D} \leftarrow \min(v_{ned,D}, 0)$ and $a_{ned,D} \leftarrow \min(a_{ned,D}, 0)$ before reprojection to body-frame derivatives.
- Explicit fixed-step lockstep integration.

3.2.2 Equations

State and kinematics:

$$y = [p_n, p_e, p_d, q_w, q_x, q_y, q_z, u, v, w, p, q, r]^T, \tag{1}$$

$$v_b = [u, v, w]^T, \quad \omega_b = [p, q, r]^T, \tag{2}$$

$$\dot{p}_{ned} = R_{bw}(q)\, v_b. \tag{3}$$

Translational and rotational dynamics:

$$f_b = [F_x, F_y, F_z]^T, \quad m_b = [M_x, M_y, M_z]^T, \tag{4}$$

$$a_b = \frac{f_b + R_{wb}(q)[0, 0, mg]^T}{m} - \omega_b \times v_b, \tag{5}$$

$$\dot{\omega}_b = I^{-1}\left(m_b - \omega_b \times I\omega_b\right). \tag{6}$$

Attitude dynamics:

$$\dot{q} = \frac{1}{2}\Omega(\omega_b)q, \tag{7}$$

where, for $\omega_b = [p, q, r]^T$ and $q = [q_w, q_x, q_y, q_z]^T$,

$$\Omega(\omega_b) = \begin{bmatrix} 0 & -p & -q & -r \\ p & 0 & r & -q \\ q & -r & 0 & p \\ r & q & -p & 0 \end{bmatrix}. \tag{8}$$

Numerical integration (applied to all state components):

$$y_{k+1} = y_k + \dot{y}_k\, \Delta t. \tag{9}$$

Quaternion normalization is applied after each integration step.

Ground-contact condition at $z_{ned} = 0$:

$$\text{if } p_d \geq 0: \tag{10}$$

$$p_d \leftarrow 0, \tag{11}$$

$$v_{ned,D} \leftarrow \min(v_{ned,D}, 0), \tag{12}$$

$$a_{ned,D} \leftarrow \min(a_{ned,D}, 0). \tag{13}$$

The clamped NED velocity and acceleration are reprojected to body-frame derivatives used by the integrator.

3.2.3 Catapult-Constrained Start Mode

When catapult start is enabled, `World` temporarily replaces free 6DOF with a rail-constrained kernel from `src/dynamics/dynamics.py` (function `rail_dynamics`), while catapult pull force is computed in `src/vehicles/common_forces/catapult_rail.py`. Let $\hat{r}_{rail}^{ned}$ be the unit rail direction and $p_{rail,0}^{ned}$ the rail start point.

Rail progress and switch condition:

$$s = (p^{ned} - p_{rail,0}^{ned})^T \hat{r}_{rail}^{ned}, \tag{14}$$

$$\text{if } s \geq L_{rail} \Rightarrow \text{switch to free 6DOF.} \tag{15}$$

Launch is additionally gated by an arm-triggered countdown in the runtime loops (CLI and ROS2):

$$t_{release} = t_{arm} + T_{countdown}, \tag{16}$$

$$\text{dynamics frozen while } t < t_{release}. \tag{17}$$

The countdown duration is configured by environment variable `SIM_CATAPULT_LAUNCH_COUNTDOWN_S` (default 3.0 s).

During constrained mode, translational motion is projected onto the rail:

$$\dot{p}^{ned} = (v^{ned} \cdot \hat{r}_{rail}^{ned})\, \hat{r}_{rail}^{ned}, \tag{18}$$

$$f_{rail} = \max\left(0,\, m\, g\, k_{pull}(L_{rail} - s)\right), \tag{19}$$

$$f_b^{rail} = R_{wb}(q) \left(f_{rail}\, \hat{r}_{rail}^{ned}\right), \tag{20}$$

$$a_b = \left(\frac{f_b + f_b^{rail} + R_{wb}(q)[0,0,mg]^T}{m} \cdot \hat{r}_{rail}^{b} \right) \hat{r}_{rail}^{b}, \tag{21}$$

$$\dot{q} = 0, \qquad \dot{\omega}_b = 0. \tag{22}$$

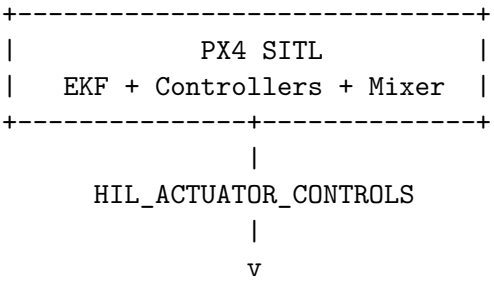
Here $k_{pull} =$ `rail_pull_max` and $\hat{r}_{rail}^{b} = R_{wb}(q)\hat{r}_{rail}^{ned}$. Attitude is held aligned to the rail until switch. So far, the catapult pull model is assumed linear with rail distance ($\propto (L_{rail} - s)$) and clamped to non-negative force.

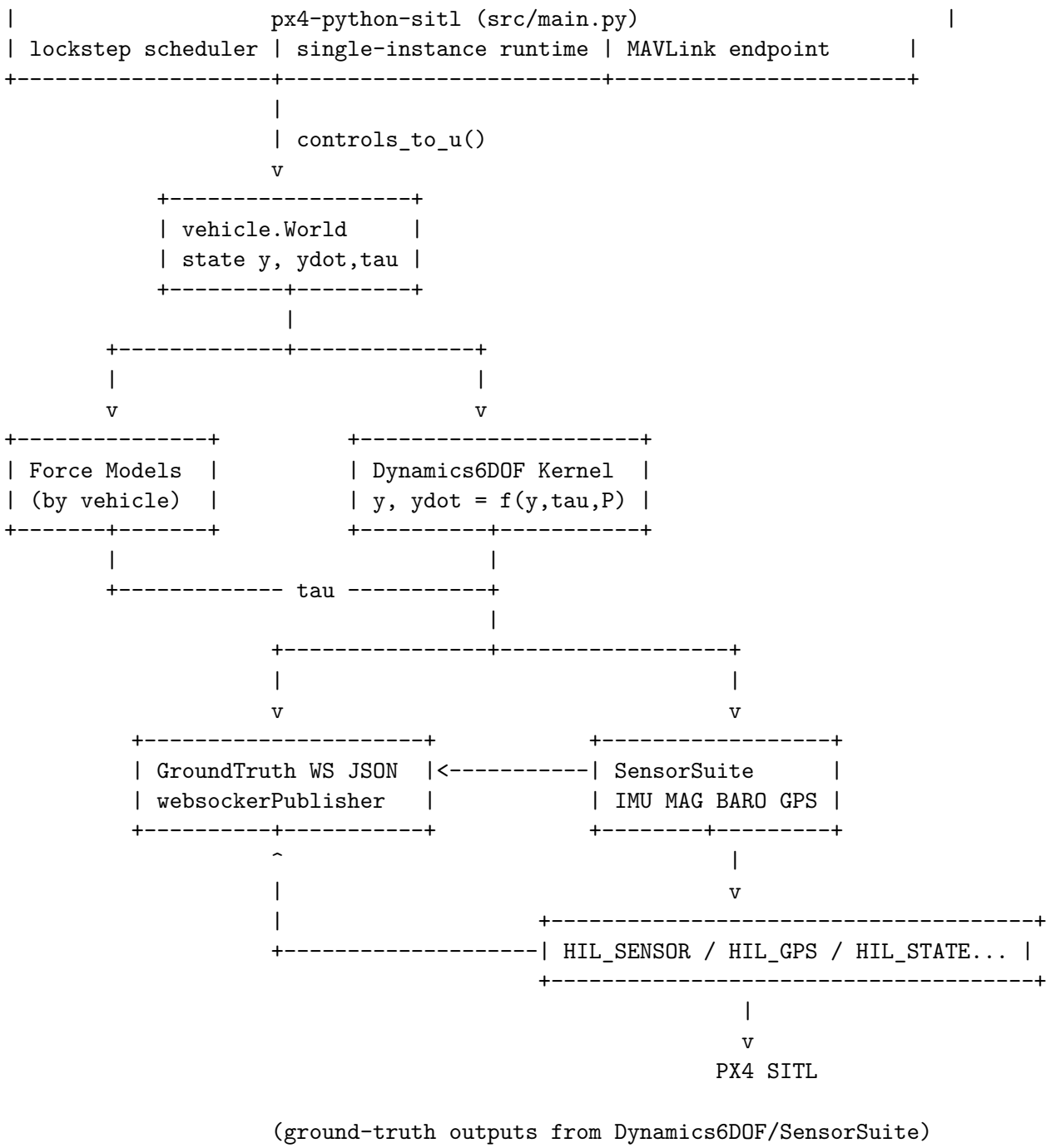
3.3 Program Architecture

The 6DOF model above defines the rigid-body state propagation used by the simulator core. Vehicle-specific force and moment generation is documented in the *Vehicle Models* chapter, where each vehicle contributes its own force-model set through the catalog.

Sensor synthesis is handled by `SensorSuite` and documented in the *Sensor Models* chapter, including IMU, magnetometer, barometer, GPS, and differential-pressure/airspeed-related channels.

Communication of simulated sensor and state outputs to PX4 (and integration hooks for external tools) is documented in the *MAVLink and External Communication* chapter.





4 Sensor Models

Sensor synthesis is implemented in `src/sensors/sensors.py`. Noise is enabled by default for IMU, magnetometer, GPS, and barometer.

4.1 IMU (Accelerometer + Gyroscope)

The IMU model in `src/sensors/imu.py` converts state and state derivative into specific-force and angular-rate measurements with LPF, bias dynamics, and additive noise.

Readable Derivation

1) Frames and core inertial quantities

$$a_{imu,in} = [\dot{u}, \dot{v}, \dot{w}]^T + \omega_b \times v_b, \tag{23}$$

$$g_b = R_{wb}(q)[0, 0, -g]^T, \tag{24}$$

$$f_{spec} = a_{imu,in} + g_b. \tag{25}$$

The $+\omega_b \times v_b$ term is required because $[\dot{u}, \dot{v}, \dot{w}]^T$ is a derivative in a rotating body frame. Using the transport theorem,

$$\left(\frac{dv_b}{dt}\right)_{inertial} = \left(\frac{dv_b}{dt}\right)_{body} + \omega_b \times v_b, \tag{26}$$

and from 6DOF translational dynamics

$$a_b = \frac{f_b + R_{wb}(q)[0, 0, mg]^T}{m} - \omega_b \times v_b, \tag{27}$$

we get

$$a_b + \omega_b \times v_b = \frac{f_b + R_{wb}(q)[0, 0, mg]^T}{m}, \tag{28}$$

which is exactly the force-per-mass term required by the IMU specific-force model. Here specific force means non-gravitational acceleration measured by the accelerometer, i.e. acceleration due to contact/propulsive/aerodynamic forces per unit mass.

Characteristic values (current parameterization) The table symbols are tied to the IMU equations through

$$f_{imu} = \frac{1}{\Delta t}, \tag{29}$$

$$\tau_a = \frac{1}{2\pi f_{c,acc}}, \quad \tau_g = \frac{1}{2\pi f_{c,gyro}}, \tag{30}$$

$$\sigma_{a,d} = \frac{\sigma_{acc}}{\sqrt{\Delta t}}, \quad \sigma_{\omega,d} = \frac{\sigma_{gyro}}{\sqrt{\Delta t}}. \tag{31}$$

Variable	Description	Value	Unit
f_{imu}	update rate	250	Hz
$f_{c,acc}$	accelerometer LPF cutoff	120	Hz
$f_{c,gyro}$	gyroscope LPF cutoff	120	Hz
σ_{acc}	accelerometer noise density	4.0×10^{-3}	$\mathrm{m/s^2/\sqrt{Hz}}$
σ_{gyro}	gyroscope noise density	3.3937×10^{-4}	$\mathrm{rad/s/\sqrt{Hz}}$
rw_{acc}	accelerometer bias random walk	6.0×10^{-3}	$\mathrm{m/s^2\sqrt{Hz}}$
rw_{gyro}	gyroscope bias random walk	3.8785×10^{-5}	$\mathrm{rad/s\sqrt{Hz}}$
τ_{ca}	accelerometer bias correlation time	300	s
τ_{cg}	gyroscope bias correlation time	1000	s
a_{max}	accelerometer range	$\pm 18g$	$\mathrm{m/s^2}$
ω_{max}	gyroscope range	± 2000	deg/s

Flow chart

```
[Input] y, ydot, q
    |
    v
[Core]  a_imu_in -> f_spec
        ([u_dot,v_dot,w_dot] + omega x v) + g_b
    |
    v
[Model] LPF -> bias/noise -> saturation
    |
    v
[Output] HIL_SENSOR.accel + HIL_SENSOR.gyro

Trigger: SensorSuite.update() on each world update tick.
Rate:    nominal f_imu = 250 Hz (model dt fallback).
Gate:    none (no next_update_us gate in IMU class).
```

2) Sensor shaping (LPF + bias + noise)

$$f_{lpf}(k) = f_{lpf}(k-1) + \alpha_a \left(f_{spec}(k) - f_{lpf}(k-1)\right), \tag{32}$$

$$\omega_{lpf}(k) = \omega_{lpf}(k-1) + \alpha_g \left(\omega_b(k) - \omega_{lpf}(k-1)\right), \tag{33}$$

$$\alpha_{a,g} = \frac{\Delta t}{\tau_{a,g} + \Delta t}, \tag{34}$$

$$b_a(k) = \phi_a b_a(k-1) + \eta_{ba}(k), \quad \phi_a = \exp\left(-\frac{\Delta t}{\tau_{ca}}\right), \tag{35}$$

$$b_g(k) = \phi_g b_g(k-1) + \eta_{bg}(k), \quad \phi_g = \exp\left(-\frac{\Delta t}{\tau_{cg}}\right). \tag{36}$$

With implementation-consistent innovation scaling,

$$\eta_{ba}(k) \sim \mathcal{N}(0, \sigma_{ba,d}^2), \quad \sigma_{ba,d}^2 = -\frac{rw_{acc}^2 \tau_{ca}}{2}\left(e^{-2\Delta t/\tau_{ca}} - 1\right), \tag{37}$$

$$\eta_{bg}(k) \sim \mathcal{N}(0, \sigma_{bg,d}^2), \quad \sigma_{bg,d}^2 = -\frac{rw_{gyro}^2 \tau_{cg}}{2}\left(e^{-2\Delta t/\tau_{cg}} - 1\right). \tag{38}$$

This stage applies first-order filtering and Gauss-Markov bias processes.

3) Final measured outputs

$$a_{meas} = \text{clip}(f_{lpf} + b_a + n_a, [-a_{max}, a_{max}]), \tag{39}$$

$$\omega_{meas} = \text{clip}(\omega_{lpf} + b_g + n_g, [-\omega_{max}, \omega_{max}]). \tag{40}$$

These channels feed accelerometer and gyroscope fields in `HIL_SENSOR`.

Assumptions

- First-order LPF for both channels.
- Bias states follow first-order Gauss-Markov processes.
- Saturation is applied to final outputs.

4.2 Magnetometer

The magnetometer model in `src/sensors/magnetometer.py` rotates Earth magnetic field from NED to body frame and applies calibration and stochastic terms.

Readable Derivation

1) Field mapping from NED to body

$$H = B_0 \cos \iota, \quad Z = H \tan \iota, \quad X = H \cos \delta, \quad Y = H \sin \delta, \tag{41}$$

$$m_{ned} = [X, Y, Z]^T, \tag{42}$$

$$m_b = R_{wb}(q)\, m_{ned}, \tag{43}$$

$$m_{si} = S_{si} m_b + b_{hi}. \tag{44}$$

The local Earth field is rotated into body coordinates, then corrected with soft-iron and hard-iron terms.

2) Bias/noise and output channel

$$b_m(k) = \phi_m b_m(k-1) + \eta_{bm}(k), \quad \phi_m = \exp\left(-\frac{\Delta t}{\tau_m}\right), \tag{45}$$

$$m_{meas} = m_{si} + b_m + n_m. \tag{46}$$

Here $\eta_{bm}(k)$ is the zero-mean Gaussian innovation that drives the magnetometer bias random process, with

$$\eta_{bm}(k) \sim \mathcal{N}(0, \sigma^2_{bm,d}), \quad \sigma^2_{bm,d} = -\frac{rw^2_{mag}\tau_m}{2}\left(e^{-2\Delta t/\tau_m} - 1\right). \tag{47}$$

This channel feeds magnetometer fields in `HIL_SENSOR`.

Characteristic values (current parameterization) All symbols in the table are defined in the equations above, with $f_{mag} = 1/\Delta t$ and rw_{mag} used to build the discrete innovation variance $\sigma_{bm,d}$.

Variable	Description	Value	Unit
f_{mag}	publication rate	100	Hz
σ_{mag}	white noise density	0.4×10^{-3}	gauss/$\sqrt{\text{Hz}}$
rw_{mag}	bias random walk	6.4×10^{-6}	gauss$\sqrt{\text{Hz}}$
τ_m	bias correlation time	600	s
δ	declination default	0.0391	rad
ι	inclination default	1.1345	rad
B_0	field strength default	0.475	gauss
m_{ned}	runtime field input	$[0.21523, 0.01, 0.43]$ (Iris)	gauss

Flow chart

```
[Input] q, m_ned
    |
    v
[Core]  rotate: m_b = R_wb m_ned
    |
    v
[Model] soft/hard-iron -> bias/noise
    |
    v
[Output] HIL_SENSOR.mag

Trigger: SensorSuite.update() on each world update tick.
Rate:    nominal f_mag = 100 Hz (model dt fallback).
Gate:    none (no next_update_us gate in MagnetometerSim).
```

Assumptions

- m_{ned} is the local Earth magnetic field vector in the NED frame.
- Soft-iron and hard-iron effects are linearized as matrix plus offset.
- Per-axis noise and bias updates are independent in implementation.

4.3 Barometer

The barometer model in `src/sensors/barometer.py` converts simulated altitude into pressure with ISA relations and adds drift and noise.

Readable Derivation

1) Altitude to ideal static pressure

$$z_{local} = -p_d, \tag{48}$$
$$h_{amsl} = h_0 + z_{local}, \tag{49}$$
$$T = T_0 - L h_{amsl}, \tag{50}$$
$$P_{ideal} = P_0 \left(\frac{T}{T_0}\right)^{5.256}. \tag{51}$$

This converts simulated altitude to ideal pressure using the standard atmosphere model.

2) Drift/noise and derived barometric outputs

$$P_{meas} = P_{ideal} + n_P + d_P, \quad d_P(k) = d_P(k-1) + r_{drift}\Delta t, \tag{52}$$
$$P_{hPa} = 0.01 P_{meas}, \tag{53}$$
$$\rho_{baro} = \rho_0 \left(\frac{T}{T_0}\right)^{4.256}, \tag{54}$$
$$h_{press} = h_{amsl} - \frac{n_P + d_P}{g\rho_{baro}}, \tag{55}$$
$$q_{dyn} = q_{dyn,lp} + n_{dp}. \tag{56}$$

These channels provide static pressure, pressure altitude, and dynamic pressure used in `HIL_SENSOR`.

Airspeed Sensor Option and Differential Pressure Channel

Each vehicle parameter class can declare `has_airspeed_sensor`. This option controls the `HIL_SENSOR.fields_updated` differential-pressure capability bit in MAVLink:

- if `has_airspeed_sensor=True`, differential pressure is advertised,
- if `has_airspeed_sensor=False`, `MAV_SYS_STATUS_SENSOR_DIFFERENTIAL_PRESSURE` (bit value 16) is cleared.

The differential-pressure measurement used by the simulator is modeled as

$$v_{air,b} = v_b - w_b, \tag{57}$$
$$u_{pitot} = v_{air,b} \cdot \hat{e}_{pitot,b}, \tag{58}$$
$$q_{dyn,ideal} = \frac{1}{2}\rho_{air}\ \max(u_{pitot}, 0)^2, \tag{59}$$
$$q_{dyn,lp}(k) = q_{dyn,lp}(k-1) + \alpha_{dp}\big(q_{dyn,ideal}(k) - q_{dyn,lp}(k-1)\big), \tag{60}$$
$$\alpha_{dp} = \frac{\Delta t}{\tau_{dp} + \Delta t}, \tag{61}$$
$$q_{dyn,meas} = q_{dyn,lp} + n_{dp}, \tag{62}$$
$$n_{dp} \sim \mathcal{N}(0, \sigma_{dp}^2). \tag{63}$$

In the current default parameterization, each vehicle sets

$$\sigma_{dp} = 0.002 \text{ Pa} \tag{64}$$

via `diff_pressure_noise_std`. The default pitot axis is `pitot_axis_body=[1,0,0]` and the default LPF constant is `diff_pressure_lpf_tau_s=0.08`.

Characteristic values (current parameterization) All symbols in the table are used by the barometer equations, with $f_{baro} = 1/\Delta t$ and $n_P \sim \mathcal{N}(0, \sigma_P^2)$.

Variable	Description	Value	Unit
f_{baro}	update rate	20	Hz
σ_P	pressure noise std. dev.	1.0	Pa
σ_{dp}	differential pressure noise std. dev.	vehicle parameter (default 0.002)	Pa
r_{drift}	pressure drift rate	0.05	Pa/s
P_0	sea-level pressure	101325	Pa
T_0	sea-level temperature	288.15	K
L	lapse rate	0.0065	K/m
ρ_0	sea-level density	1.225	kg/m^3
h_0	home altitude default	488	m

Flow chart

```
[Input] p_d, v_ned
    |
    v
[Core]  z_local -> h_amsl -> ISA P_ideal
    |
    v
[Model] drift/noise -> P_meas
    |
    v
[Output] static/dynamic/altitude -> HIL_SENSOR.baro

Trigger: SensorSuite.update() on each world update tick.
Rate:    nominal f_baro = 20 Hz (model dt fallback).
Gate:    none (no next_update_us gate in BarometerSensor).
```

Assumptions

- Standard atmosphere constants are fixed.
- Drift is modeled as an accumulated bias with constant rate.
- Dynamic pressure uses speed magnitude from current simulated velocity.

4.4 GPS

The GPS model in `src/sensors/gps.py` perturbs local NED position and velocity with random walk and white noise, then reprojects position to geodetic coordinates around the configured origin.

Readable Derivation

1) Stochastic NED position states

$$\beta_i(k) = \beta_i(k-1) + w_i(k)\Delta t - \frac{\beta_i(k-1)}{\tau_{gps}}, \quad i \in \{N, E, D\}, \tag{65}$$

$$\tilde{N} = p_n + \nu_N + \beta_N, \quad \tilde{E} = p_e + \nu_E + \beta_E, \quad \tilde{D} = p_d + \nu_D + \beta_D. \tag{66}$$

Here $w_i(k)$ is the zero-mean stochastic driving term (Gaussian random walk excitation) for the GPS bias state on axis i, with $w_i \in \{w_N, w_E, w_D\}$. In code this corresponds to the sampled `random_walk[i]` used in the bias update before position reprojection. This forms noisy and bias-corrupted local position states from true simulation position.

2) Geodetic reprojection and altitude

$$x = \frac{\tilde{N}}{R_E}, \quad y = \frac{\tilde{E}}{R_E}, \quad c = \sqrt{x^2 + y^2}, \tag{67}$$

$$\varphi = \arcsin\left(\cos c \sin\varphi_0 + \frac{x \sin c \cos\varphi_0}{c}\right), \tag{68}$$

$$\lambda = \lambda_0 + \operatorname{atan2}\left(y \sin c,\, c\cos\varphi_0 \cos c - x \sin\varphi_0 \sin c\right), \tag{69}$$

$$h_{gps} = h_0 - \tilde{D}. \tag{70}$$

This maps local NED position to latitude, longitude, and altitude at the configured origin.

3) Velocity channels used by MAVLink GPS

$$\tilde{v}_N = v_{ned,N} + \xi_N, \quad \tilde{v}_E = v_{ned,E} + \xi_E, \quad \tilde{v}_D = v_{ned,D} + \xi_D, \tag{71}$$

$$v_{gps,N} = \tilde{v}_N, \quad v_{gps,E} = \tilde{v}_E, \quad v_{gps,U} = -\tilde{v}_D. \tag{72}$$

These values are used by `HIL_GPS` once the startup-delay and update gates are satisfied.

Characteristic values (current parameterization) All symbols in the table are used by the GPS equations, with $f_{gps} = 1/\Delta t$ and w_i as the bias-driving random-walk term in the β_i update.

Variable	Description	Value	Unit
f_{gps}	update rate	5	Hz
σ_{xy}	horizontal position noise density	2×10^{-4}	$\mathrm{m}/\sqrt{\mathrm{Hz}}$
σ_z	vertical position noise density	4×10^{-4}	$\mathrm{m}/\sqrt{\mathrm{Hz}}$
σ_{vxy}	horizontal velocity noise density	0.2	$\mathrm{m/s}/\sqrt{\mathrm{Hz}}$
σ_{vz}	vertical velocity noise density	0.4	$\mathrm{m/s}/\sqrt{\mathrm{Hz}}$
w_N, w_E	horizontal bias-drive process terms	from XY random walk amplitude 2.0	m/s
w_D	vertical bias-drive process term	from Z random walk amplitude 4.0	m/s
τ_{gps}	bias correlation time	60	s
R_E	Earth radius in projector	6,371,000	m
$(\varphi_0, \lambda_0, h_0)$	home origin defaults	$(47.397742°, 8.545594°, 488)$	deg,deg,m

Flow chart

```
[Input] p_ned, v_ned, t_us
    |
    v
[Gate]  if t_us < next_update_us:
          return last sample (gps_updated = false)
    |
    v
[Core]  bias/noise states -> NED->LLA reprojection
    |
    v
[Output] HIL_GPS payload (gps_updated = true)

Trigger: SensorSuite.update(), then internal next_update_us check.
Rate:    nominal f_gps = 5 Hz.
Gate:    if t_us < next_update_us, return last sample and gps_updated=false.
```

Assumptions

- Local spherical-Earth reprojection with constant R_E.
- Bias/random-walk terms are first-order correlated states.
- Position and velocity channel noise terms are independent in the model.

5 Vehicle Models

Vehicle parameters are declared per vehicle in `src/vehicles/*/parameters.py`.

5.1 Iris (Multicopter)

`IrisParameters` defines mass, inertia, motor, and passive drag terms. The currently used stable defaults are:

- $J_x = 0.029125$, $J_y = 0.029125$, $J_z = 0.055225$ kg m^2,
- motor time constant $\tau_m = 0.06$ s.

Force and Moment Models

The Iris model uses two force components:

- `IrisQuadForceModel` in `src/vehicles/iris/forces.py`: active rotor thrust and reaction torque,
- `PassiveSphereAeroForceModel` in `src/vehicles/common_forces/passive_sphere_aero.py`: passive drag.

Rotor moments include arm-induced moments, spin-direction reaction torque, and rotor gyroscopic coupling.

5.2 X8 (Fixed-Wing)

`X8Parameters` provides aerodynamic derivatives for lift, drag, side force, and moments, plus propulsive and geometric terms.

Force and Moment Models

The X8 model uses `WingX8ForceModel` in `src/vehicles/x8/forces.py`. It computes aerodynamic forces from angle of attack/sideslip and aerodynamic derivatives, then adds propulsion force and moment.

5.3 TS06 (VTOL)

`Ts06Parameters` defines physical, sensor, aerodynamic, and motor-propulsion properties of the VTOL quad-tailsitter vehicle model.

Force and Moment Models

The TS06 model uses `Ts06ForceModel` in `src/vehicles/ts06/forces.py`, which implements:

- **CFD-based Aerodynamics Lookup**: Performs lookup in a multi-dimensional CFD-derived table (`aerodynamics_table.csv` via `aero_lookup.py`) using low-pass filtered angle of attack (α) and sideslip (β) within a nominal envelope ($\pm 15^\circ$). Dynamic rate damping moments are computed using reference dimensions and aerodynamic rate-damping coefficients (C_{l_p}, C_{m_q}, C_{n_r}).
- **Four-Motor Propulsion**: Active thrust, reaction torque, and geometric moment arm for four vertical/forward-facing brushless DC motors, accounting for local motor position, rotation direction (CW/CCW), and forward speed seen by each propeller.

5.4 How to Add a New Vehicle

Add a new folder under `src/vehicles/` (for example `my_uav/`) with:

- `parameters.py`: mass, inertia, sensor terms, and optional rail launch defaults,
- `forces.py`: vehicle-specific force/moment models,
- `initial_state.py`: startup state builder,
- `definition.py`: `make_parameters()`, `make_force_models(parameters)`, `make_initial_state(config`

Then register the vehicle in `src/vehicles/vehicle`$_c$`atalog.pybyaddingone`VEHICLES`entry`.

5.5 Vehicle Options

5.5.1 Catapult Start Parameters (Per Vehicle)

Catapult behavior is configured in each vehicle parameter class (not through environment variables). Common fields are:

- rail_launch_enabled (bool),
- rail_dir_ned (unit direction in NED),
- rail_start_ned (rail origin in NED),
- rail_length (switch distance),
- rail_pull_max (catapult pull scaling),
- left_rail (runtime state latch).

If enabled, World starts in constrained rail mode and switches automatically to free 6DOF when rail distance exceeds rail_length.

6 MAVLink and External Communication

6.1 MAVLink Lockstep Bridge

src/main.py connects to PX4 via MAVLink (TCP endpoint), receives actuator commands, advances lockstep simulation, and publishes HIL messages:

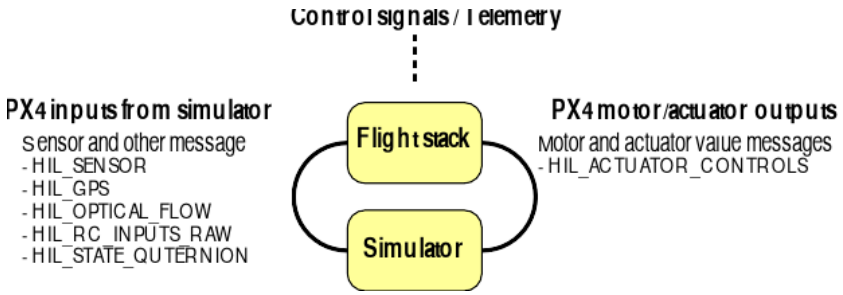


Figure 2: PX4 interface graphic (source asset: ../media/px4.svg, rasterized to PNG for pdflatex compatibility).

- HIL_SENSOR at simulation update rate,
- HIL_GPS on GPS update events,
- HIL_STATE_QUATERNION when requested by MAV_CMD_SET_MESSAGE_INTERVAL,
- SYSTEM_TIME and periodic heartbeat.

Lockstep Message Sequence

```
Init defaults in main.py:
  sim_time_us = 0
  now_ms = sim_time_us // 1000
  RATE_HZ = 250, DT_US = 4000
  slow_down_counter = 0, CHECK_FACTOR = 2
  hil_state_interval_us = -1      (HIL_STATE_QUAT disabled)
```

```
  gps_start_time_us = 1_000_000us (1 s delay)

PX4                    Bridge(main.py)                World/Sensors
 |                           |                              |
 |-- HEARTBEAT ------------> |                              |
 |-- HIL_ACTUATOR_CTRL ---> |                              |
 |-- COMMAND_LONG (opt) --> |                              |
 |                           |-- parse RX MAVLink ---------->|
 |                           |<-- controls + arming ---------|
 |                           |                              |
 |                           | local standalone/master mode: |
 |                           |   sim_time_us += DT_US       |
 |                           |   ioOnlyRun = (slow_down_counter % 2 != 0)
 |                           |   if ioOnlyRun: I/O-only cycle
 |                           |      (skip full dynamics step)
 |                           |   else world.update(...) ----->|
 |                           |        <- y, ydot, sensors ----|
 |                           |   slow_down_counter += 1       |
 |                           |                              |
 |<-- HIL_SENSOR ---------- | from SensorSuite publish path  |
 |<-- HIL_STATE_QUAT* ----- | from state y                   |
 |<-- HIL_GPS** ----------- | from SensorSuite GPS output    |
 |<-- SYSTEM_TIME (1Hz) ----|                              |
 |<-- HEARTBEAT (1Hz) ------|                              |
 |                           |-- sleep(1/RATE_HZ), next loop |

* HIL_STATE_QUAT default: OFF (hil_state_interval_us = -1)
  Enabled by MAV_CMD_SET_MESSAGE_INTERVAL for msg id 115.

** HIL_GPS default send condition:
   sim_time_us >= gps_start_time_us AND gps_updated == true.

Note: sim_time_us is incremented before ioOnlyRun is evaluated, so it advances
      even during I/O-only cycles.
```

Simulation-step timing rule in standalone/master mode:

$$t_{k+1} = t_k + \Delta t, \quad \Delta t = \frac{1}{\texttt{RATE\_HZ}}. \tag{73}$$

6.2 Ground Truth WebSocket

src/networking/websocket$_p$ublisher.pystreamsJSONground−truthstateforexternalvisualizersandanalys

7 Python Installation and Interaction

7.1 Install

This repository is installable as a Python package (see pyproject.toml).

python -m pip install -e .

The console entry point is:

px4-python-sitl

7.2 Run from CLI

Typical usage is to start the bridge and connect it to PX4 SITL over MAVLink.

```
px4-python-sitl --help
python src/main.py --help
```

7.3 Interact from Python

You can import simulation modules and run/update components programmatically.

```
from vehicle.world import World

world = World(vehicle_model="iris")

# set controls/wind, advance lockstep, read state/sensors
world.set_controls([0.0, 0.0, 0.0, 0.0])
world.set_wind([0, 0, 0, 0, 0, 0])
out = world.update(t_us=4000, paused=False)
print(out["y"])         # state
print(out["sensors"])   # synthesized sensors
```

7.4 External Program Interfaces

- MAVLink HIL I/O via src/main.py.
- Ground-truth WebSocket via src/networking/websocket$_publisher.py$.

8 Build and Use

The document can be built with:

```
make -C docs
```

or directly:

```
pdflatex -output-directory docs docs/architecture.tex
```